A4 — Policy Gradient & Actor-Critic: task, goal, and how it was solved

What phase A4 asks, why parameterising the policy directly is a different kind of answer from everything before it, and what the runs showed. The lab notebook with the numbers is `experiments/a4_policy_gradient/README.md`.

> **At a glance**
>
> - On CartPole, REINFORCE (333) and REINFORCE with a baseline (451) beat A3's DQN (250), with a smaller spread across seeds.
> - A baseline is free in expectation and worth a lot in practice: it cuts advantage spread by 30%.
> - GAE λ trades bias for variance exactly as the estimator predicts. λ = 0 has the lowest variance and fails (20.8); the best is λ = 0.99 (479).
> - A2C's apparent thousand-point win on LunarLander was mostly a confound, and a control caught it.

---

1. The task

`ladder.md` states three items:

1. REINFORCE on CartPole; compare variance and efficiency against the A3 DQN run.
2. Add a learned baseline and GAE; sweep λ and observe the bias/variance tradeoff.
3. A2C on LunarLander; compare stability against REINFORCE-with-baseline.

2. The goal

Everything up to here learned a *value* and recovered a policy from it by taking an argmax. This phase parameterises the policy itself, $\pi_\theta(a \mid s)$, and improves it by gradient ascent on expected return.

The **policy-gradient theorem** says that gradient is computable from samples without differentiating through the environment:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t]$$

Read it as an instruction: push up the log-probability of an action in proportion to the return that followed it. Nothing about the environment's dynamics appears, which is why it works on a black box.

Where that comes from. A trajectory $\tau = (s_0, a_0, s_1, \dots)$ has probability

$$p_\theta(\tau) = \rho(s_0) \prod_t \pi_\theta(a_t \mid s_t)\, P(s_{t+1} \mid s_t, a_t)$$

Differentiating $J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[R(\tau)]$ directly is awkward because the *distribution* depends on θ. The log-derivative identity $\nabla p = p \nabla \log p$ converts it into an expectation that can be sampled:

$$\nabla_\theta J = \int \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau = \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, R(\tau)\, d\tau = \mathbb{E}\left[\nabla_\theta \log p_\theta(\tau)\, R(\tau)\right]$$

Now take the log of the trajectory probability:

$$\log p_\theta(\tau) = \log \rho(s_0) + \sum_t \log \pi_\theta(a_t \mid s_t) + \sum_t \log P(s_{t+1} \mid s_t, a_t)$$

The initial-state distribution and the transition kernel do not depend on θ, so they differentiate to zero. This is the whole trick — the environment drops out exactly, no model required:

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

One more refinement gives the form actually implemented. An action cannot affect rewards that arrived before it, and for $t' < t$, $\mathbb{E}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\, r_{t'}] = 0$ by the same argument that kills the baseline in section 3. Dropping those terms replaces the full-trajectory return $R(\tau)$ with the reward-to-go G_t — still unbiased, strictly lower variance. That is the estimator in the box above, and the first variance reduction of the phase, applied before the experiment even begins.

Three things change relative to A3, and it is worth being precise about which are gains:

- **The max operator is gone.** A3 item 2 measured the bias it induces; here the policy is improved directly, so there is nothing to overestimate.
- **The action space stops mattering.** $\arg\max_a Q(s,a)$ needs a finite enumerable set; π_θ does not. This is what makes A5 (continuous control) and all of Part B possible.
- **The policy is stochastic by construction**, so exploration is built into the object being learned rather than bolted on as ε.

The cost is **variance**. G_t is a single sampled return, and the estimator credits every action in an episode with all of it — including the reward that arrived before the action was taken and had nothing to do with it. Items 2 and 3 are both about reducing that variance without introducing so much bias that the gradient points somewhere else.

3. Why a baseline is free (item 2)

Subtract any function of state from the return:

$$\nabla_\theta J(\theta) = \mathbb{E}\big[\nabla_\theta \log \pi_\theta(a \mid s)\, \big(G_t - b(s_t)\big)\big]$$

This leaves the gradient unchanged, because for any b depending only on state,

$$\mathbb{E}_{a\sim\pi}[\nabla_\theta \log \pi_\theta(a \mid s)\, b(s)] = b(s) \nabla_\theta \sum_a \pi_\theta(a \mid s) = b(s)\, \nabla_\theta 1 = 0$$

So the baseline is *free* in expectation and can only change the variance. The natural choice is $b(s) = V(s)$, which turns the return into an advantage $A(s,a) = Q(s,a) - V(s)$: how much better this action was than the state's average, rather than how good the state happened to be.

Is V the best choice? Not quite. Minimising the variance of the scalar estimator $g = \nabla_\theta \log \pi_\theta \cdot (G - b)$ over b is a one-dimensional least-squares problem, and setting $\partial \mathrm{Var}(g)/\partial b = 0$ gives

$$b^\star = \frac{\mathbb{E}\left[(\nabla_\theta \log \pi_\theta)^2 G\right]}{\mathbb{E}\left[(\nabla_\theta \log \pi_\theta)^2\right]}$$

a gradient-magnitude-weighted average of the return rather than a plain one. In practice $V(s)$ is used anyway: it is close to optimal, it is a quantity worth having for its own sake, and $b^\star$ is per-parameter, so using it exactly would mean a different baseline for every component of θ. Worth knowing that the standard choice is a convenient approximation, not the optimum.

GAE then interpolates between two ways of estimating that advantage. With $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$,

$$\hat{A}_t^{\mathrm{GAE}(\lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \, \delta_{t+l}$$

At $\lambda = 0$ this is the single residual δ_t: minimum variance, but biased by whatever error V carries. At $\lambda = 1$ the sum telescopes into $G_t - V(s_t)$: the Monte Carlo advantage, unbiased but carrying the full variance of the sampled return. The claim to test is that neither endpoint is best.

The telescoping, since it is the reason $\lambda = 1$ is unbiased. Substituting $\delta_{t+l} = r_{t+l} + \gamma V(s_{t+l+1}) - V(s_{t+l})$ at $\lambda = 1$:

$$\sum_{l \geq 0} \gamma^l \delta_{t+l} = \sum_{l \geq 0} \gamma^l r_{t+l} + \sum_{l \geq 0} \left(\gamma^{l+1} V(s_{t+l+1}) - \gamma^l V(s_{t+l})\right) = G_t - V(s_t)$$

The second sum collapses because consecutive terms cancel, leaving only $-V(s_t)$. Every appearance of V except the baseline term vanishes, which is why no amount of critic error can bias it. At $\lambda < 1$ the cancellation is incomplete and the surviving V terms are exactly the bias — so λ interpolates between "trust the critic" and "trust the samples", and its optimum sits wherever the critic's error and the return's variance balance.

An equivalent reading: GAE is an exponentially-weighted average of the k-step advantage estimators $\hat{A}_t^{(k)} = \sum_{l<k} \gamma^l r_{t+l} + \gamma^k V(s_{t+k}) - V(s_t)$,

$$\hat{A}_t^{\mathrm{GAE}(\lambda)} = (1-\lambda) \sum_{k \geq 1} \lambda^{k-1} \hat{A}_t^{(k)}$$

which is the same weighting TD(λ) applies to n-step returns — the bias/variance knob is the same object in both.

4. Why A2C needs parallel environments (item 3)

A3 used a replay buffer to decorrelate its training batches. Policy gradients cannot: the estimator above is an expectation **under the current policy**, so data from an older policy is simply the wrong distribution. Replay is unavailable by construction.

A2C's answer is to decorrelate *across environments instead of across time* — step N independent copies simultaneously and take one transition from each. The batch is decorrelated, and every sample is on-policy. This is why on-policy methods are described as needing parallelism where off-policy methods need memory, and it is the same tradeoff that reappears in B1: PPO's clipping exists to squeeze a few extra gradient steps out of each batch precisely because a fresh batch is expensive.

5. What was done

`pg.py` implements the family as one estimator with flags — `baseline=False` is REINFORCE, `baseline=True` adds a critic, `gae_lambda` selects the advantage estimator — so the arms differ only where the phase says they differ.

Decisions worth recording:

- **Item 1 reuses A3's exact evaluation protocol**: CartPole-v1, 30k environment steps, greedy evaluation every 2000, five seeds. "Comparable" should mean numerically comparable, not comparable in spirit. A3's reference numbers are recorded as constants in the script so the comparison does not depend on a gitignored run directory.
- **The critic gets its own optimizer and many gradient steps per batch.** With a single shared update the value function never fits — the first attempt here reached an explained variance of 0.0, meaning the "baseline" was subtracting noise and item 2 would have measured nothing. A policy gradient takes one on-policy step per batch; a critic is a supervised regression and needs more.
- **Advantage spread is recorded before normalisation.** Normalising advantages is standard and helps optimisation, but it rescales exactly the quantity λ controls. Measuring after normalisation would erase the effect.
- **Truncation is bootstrapped, termination is not** — the same distinction as A3, and equally easy to get wrong here.
- **Greedy evaluation takes the mode of the policy**, so the evaluation number is not inflated by exploration and is comparable with A3's.

6. Results

Tables in the phase README. The parts that change how the phase should be read:

Policy gradients beat DQN here, including on variance — REINFORCE 333.4, with a baseline 450.9, against A3's DQN at 250.1, and with smaller seed spread (±119, ±98 against ±151). That inverts the usual framing, so it needs care. "High variance" describes the

gradient estimator, and that variance is real and visible: the advantage spread falls from 22.6 to 15.9 when the baseline is added. DQN's spread is a different phenomenon — divergence, which A3 measured directly. Both systems are lightly tuned at an equal budget, so this compares these implementations at 30k steps, not the algorithm families.

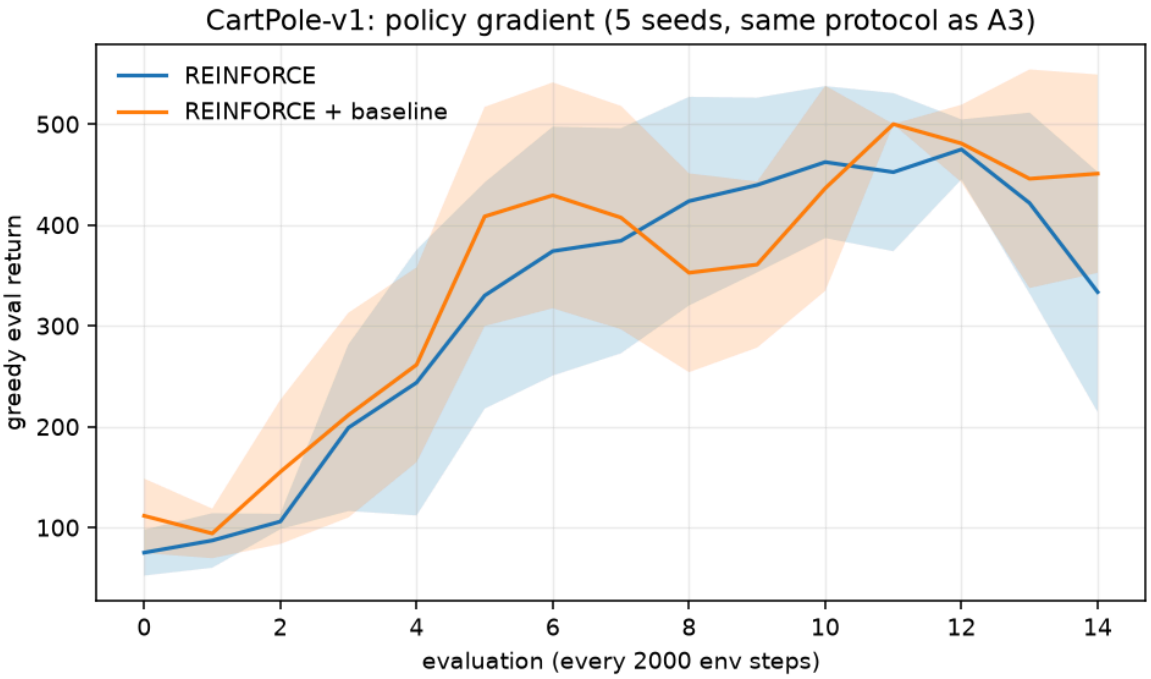


Figure — CartPole, 5 seeds, the same protocol as A3: REINFORCE with and without a learned baseline.

The baseline is free in theory and substantial in practice. Section 3 proves it cannot change the gradient in expectation; the run shows it cutting advantage spread 30% while raising return from 333 to 451 and retention from 0.68 to 0.90.

The λ sweep exhibits both sides of the tradeoff. Advantage spread is monotone in λ — $2.20, 2.95, 5.53, 7.76, 12.54, 15.86$ — exactly as the estimator predicts, since larger λ admits more of the sampled return. The bias side appears at the bottom: $\lambda = 0$ **scores** 20.8, near random, *despite having the lowest variance of any arm*. The one-step residual is only a good advantage estimate when V is accurate, and the critic's explained variance there is 0.05 — so the cleanest signal in the sweep is almost entirely an estimate of the critic's own error. Minimum variance is worthless when the estimand is wrong. The optimum is interior, at $\lambda = 0.99$ (478.8 ± 26.0, also the tightest across seeds).

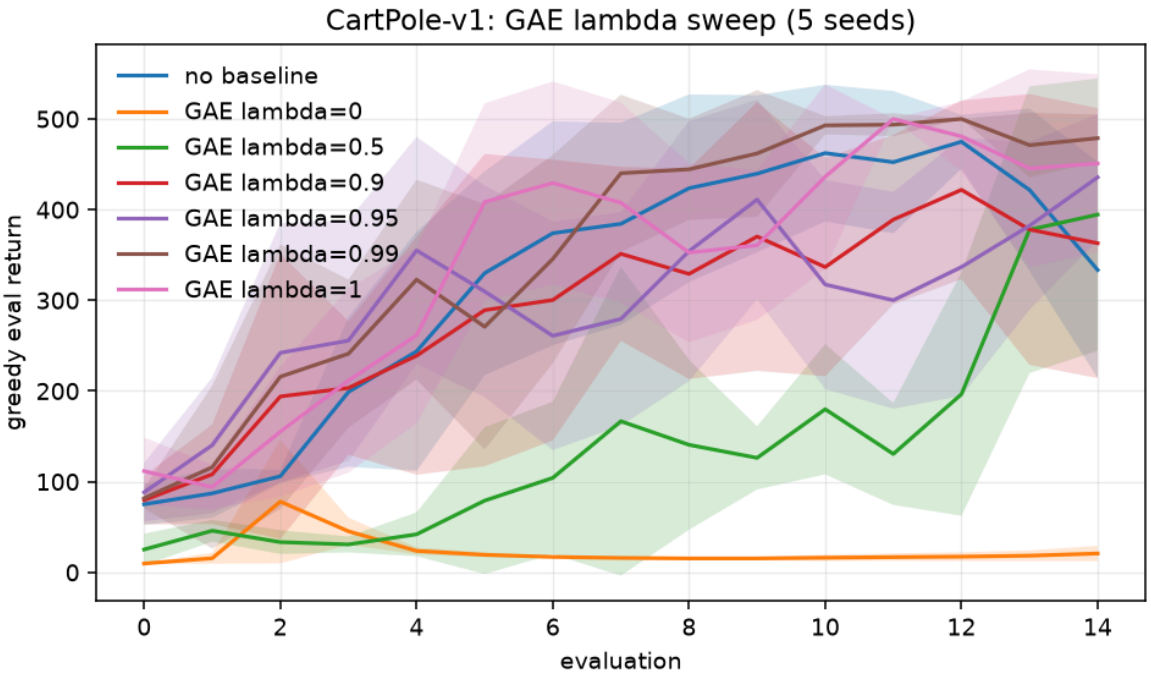


Figure — GAE λ sweep on CartPole, 5 seeds. λ = 0 barely learns despite the lowest-variance advantages; the curves improve as λ approaches 1.

Item 3's headline was mostly a confound, and a control caught it. A2C appeared to beat episodic REINFORCE by a thousand points on LunarLander; but shrinking REINFORCE's batch from 4000 to 1000 — same algorithm, same environment steps, 4× the updates — recovers about 900 of them. A2C still wins on what remains (27.6 against −75.6, with a quarter of the thrash), and it is the only arm reaching positive return, but the effect is an order of magnitude smaller than the raw table implied.

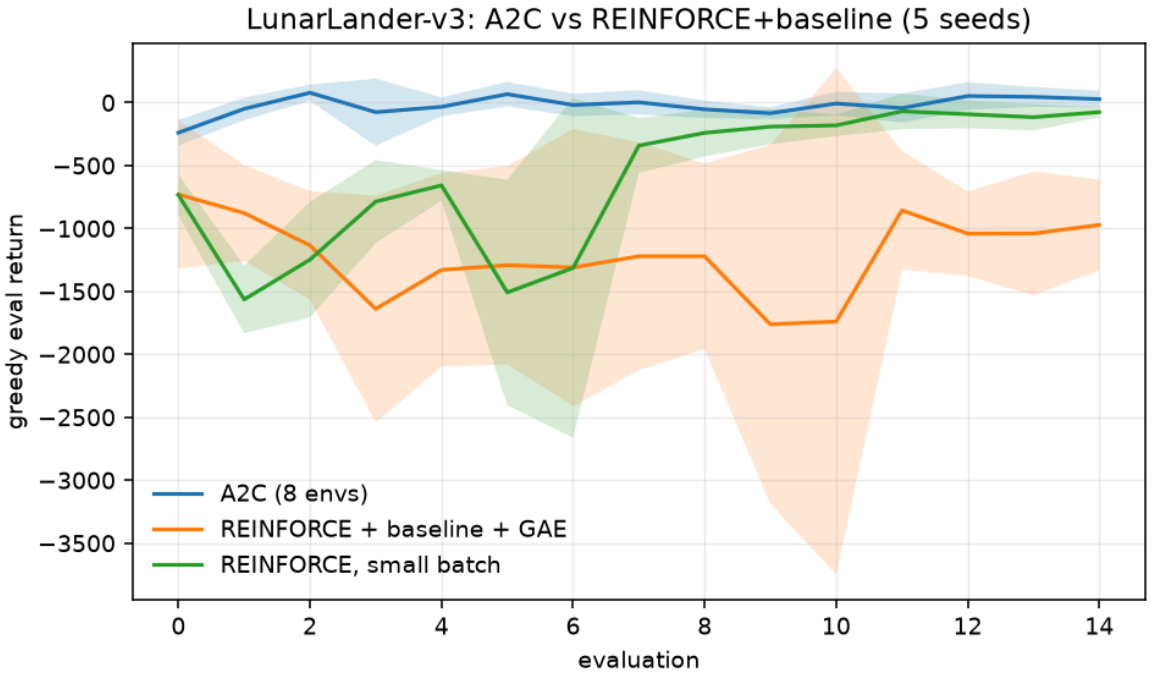


Figure — LunarLander, 5 seeds: A2C with 8 environments, REINFORCE with baseline and GAE, and the small-batch REINFORCE control.

The general lesson is about the axis, not the algorithms. Holding *environment steps* fixed is the right comparison when environment interaction is the scarce resource, and it is the standard axis in RL — but it silently lets methods differ by 100× in gradient steps taken from that data. A comparison that does not report both is measuring something other than what it claims.

7. What this sets up

- **A5** needs the action space to be continuous, which the argmax in A3 could not handle and the policy parameterisation here can. DDPG's actor is exactly the "learn the maximising action" idea made explicit.
- **B1** is this phase at scale: PPO is a policy gradient with a trust region, and its KL term is a baseline-like correction that keeps the estimator valid against a reference distribution rather than against a value function.
- **B3's GRPO** replaces the learned $V(s)$ derived here with the mean score of a group of completions — a Monte Carlo baseline instead of a fitted one, which is why the group-size sweep is a variance measurement.

8. Where these items come from

item	primary source	why it was picked
1 — REINFORCE	Williams 1992, the paper that named it; S&B §13.3	It is the policy-gradient theorem with nothing added, so it isolates the estimator before any variance reduction obscures it. Comparing against A3 on the same task is what makes the "high variance" claim a measurement rather than folklore.
2 — baseline and GAE	S&B §13.4 for the baseline; Schulman et al. 2016 for GAE	The baseline is the one variance reduction that is provably free, and GAE is the standard knob that spans from TD to Monte Carlo — one sweep covers the whole bias/variance axis instead of two separate algorithms.
3 — A2C	Mnih et al. 2016, which introduced A3C; A2C is its synchronous variant	It is the smallest change that makes policy gradients practical on a harder task, and the mechanism — parallel environments in place of replay — is the on-policy/off-policy distinction from A2 showing up as a systems decision.

The **lab** pairing (`Practical_RL` week06_policy_based) is the ladder's own and covers REINFORCE and advantage actor-critic, matching items 1 and 3.

The **goal statement** in section 2 is quoted from `ladder.md`. Section 4's framing of parallel environments as "the on-policy substitute for replay" is this repo's, not a claim from the A3C paper.